Recursion Exercises

October 2026

This is not an official EProg document and has therefore not been checked for correctness by the EProg team. Its purpose is to practice recursion in the exercise session.

Short Exercises

You are given the class `RecTasks`, which contains several methods for practicing recursive thinking. All methods must be implemented **recursively**. Loops are not allowed. Where necessary, you may define a *helper method* to keep track of intermediate state (e.g., the current index or partial results). When processing strings, consistently use `substring` instead of index arithmetic.
You may assume that no input is `null`. All other properties (e.g., empty strings or arrays) must be checked. A method is only correct if all of the described conditions are met.

Exercise 1: Sum of Elements

Write a method

```
int sum(int[] xs)
```

that computes the sum of all elements of an array.
Examples: `[]` → 0
`[7]` → 7
`[2, 5, 1]` → 8
Hint: Use a helper method that carries the current index along. This lets you build up the sum step by step without a loop.

Exercise 2: Counting Characters

Implement

```
int count(char c, String s)
```

which counts how often the character `c` occurs in the string `s`. Use `s.substring(1)` to recurse over the rest of the string.
Examples:
`count('a', "banana")` → 3
`count('x', "")` → 0

Exercise 3: Sorted Array

Implement

```
boolean isSorted(int[] xs)
```

which checks whether the array `xs` is sorted in **non-decreasing order**.
Examples:
`[]` → `true`
`[5]` → `true`

`[1,2,2,5]` → `true`
`[2,1]` → `false`
Hint: Use a helper method with an index parameter to compare the next pair of elements.

Exercise 4: Removing Repetitions

Write a method

```
String dedupRuns(String s)
```

that reduces consecutive identical characters to a single character.
Examples:
`"aabbcca"` → `"abca"`
`"mississippi"` → `"misisipi"`
Hint: Use a helper method that stores the last character in order to detect repetitions. Work with `substring`, not with indices.

Exercise 5: Checking Brackets (Balanced?)

Write a method

```
boolean isBalanced(String s)
```

that checks whether the string is correctly balanced with respect to curly braces `{ }`.
Note:

- Every opening brace `{` must be closed later, and every closing brace `}` must belong to a previously opened `{`.
- If any of the characters `(`, `)`, `[`, `]`, `<`, `>` occurs, the result is `false`.
- All other characters may be ignored.

Examples:
`""` → `true`
`"{}"` → `true`
`"{{}}"` → `true`
`"{a{}}"` → `true`
`"({a})"` → `false`
`"}{"` → `false`
`"{"` → `false`
Hint: Use a helper method.